ĐẠI HỌC QUỐC GIA HÀ NỘI – TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Pretrained Transformers & BERT Fine-Tuning

From Self-Attention Architecture to Downstream Task Adaptation

---

Báo cáo viên: Phạm Ngọc Hải

Thời gian: 2026-11-01

Table of contents I

PART I: INTRODUCTION TO SEQUENCE MODELING

PART II: THE TRANSFORMER ARCHITECTURE

PART III: PRETRAINING PARADIGMS AND BERT

PART IV: FINE-TUNING BERT FOR DOWNSTREAM TASKS

PART V: TRANSFORMER VARIANTS & EVOLUTION

PART VI: PRACTICAL HANDS-ON GUIDANCE

PART VII: EVALUATION AND DIAGNOSTICS

PART VIII: BEYOND TEXT - MULTIMODAL & ADVANCED APPLICATIONS

PART IX: COMPREHENSIVE RECAP & FUTURE DIRECTIONS

PART X: ADVANCED TOPICS AND RESEARCH FRONTIERS

PART I: INTRODUCTION TO SEQUENCE MODELING

Slide 1: The Evolution of Sequence Models

- Traditional NLP heavily relied on **Rule-based systems** and **Statistical Language Models (n-grams)**.
- **Recurrent Neural Networks (RNNs)** introduced sequential state representation.
- Transition from static word embeddings (Word2Vec, GloVe) to contextualized embeddings (ELMo, BERT).
- Shift from sequential computation to parallelized matrix computations via Attention mechanism.

Slide 2: Limitations of Traditional Word Embeddings

- **Word2Vec & GloVe** assign a single static vector to each token in the vocabulary.
- Polysemy ambiguity:
  - *Example:* “Bank of the river” vs. “Investment bank”.
  - Both instances of “bank” share the identical representation vector $\mathbf{v}_{\text{bank}}$.
- Incapable of dynamically encoding surrounding contextual nuances.

Slide 3: The Recurrent Era (RNNs and LSTMs)

- **RNNs** process input sequentially:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$$

- Hidden state $\mathbf{h}_t$ acts as a memory buffer across time steps $t = 1, \ldots, T$.
- Bottleneck: Information from early time steps decays rapidly over long sequences.

Slide 4: Vanishing & Exploding Gradients in RNNs

- Backpropagation Through Time (BPTT) involves repeated matrix multiplication by $\mathbf{W}_{hh}$.
- Gradient calculation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{t=2}^{T} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

- If eigenvalues of $\mathbf{W}_{hh} < 1$, gradients vanish exponentially.
- If eigenvalues of $\mathbf{W}_{hh} > 1$, gradients explode.

Slide 5: Gated Architectures (LSTM & GRU)

- **LSTM (Long Short-Term Memory)** introduces a dedicated Cell State $\mathbf{C}_t$ and gating mechanisms:
  - Input Gate $\mathbf{i}_t$, Forget Gate $\mathbf{f}_t$, Output Gate $\mathbf{o}_t$.
- **GRU (Gated Recurrent Unit)** simplifies LSTM into Reset $\mathbf{r}_t$ and Update $\mathbf{z}_t$ gates.
- *Remaining Bottleneck:* Sequential processing prevents parallel execution on Modern GPUs.

Slide 6: Seq2Seq and the Encoder-Decoder Paradigm

- Proposed by Sutskever et al. (2014) for Neural Machine Translation (NMT).
- **Encoder:** Compresses entire input sequence $\mathbf{X} = (x_1, \ldots, x_N)$ into a fixed-length context vector $\mathbf{c}$.
- **Decoder:** Generates target sequence $\mathbf{Y} = (y_1, \ldots, y_M)$ conditioned on $\mathbf{c}$.
- Loss of information occurs when compressing long passages into a single vector $\mathbf{c}$.

Slide 7: The Attention Mechanism Breakthrough

- Bahdanau et al. (2014) & Luong et al. (2015).
- Replaces fixed vector $\mathbf{c}$ with dynamic context vectors $\mathbf{c}_i$ calculated at each decoding step.
- Soft alignment weights α_{ij}:

$$\mathbf{c}_i = \sum_{j=1}^{N} \alpha_{ij} \mathbf{h}_j, \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{N} \exp(e_{ik})}$$

- Decouples alignment from fixed memory constraints.

Slide 8: Why Attention Replaced Recurrence

- **Computational Complexity per Layer:**
  - Recurrent: $\mathcal{O}(N \cdot d^2)$
  - Self-Attention: $\mathcal{O}(N^2 \cdot d)$
- **Maximum Path Length:**
  - Recurrent: $\mathcal{O}(N)$ sequential steps.
  - Self-Attention: $\mathcal{O}(1)$ direct operations between any pair of tokens.
- Enables full GPU training parallelization across sequence length N.

PART II: THE TRANSFORMER ARCHITECTURE

Slide 9: “Attention Is All You Need” (Vaswani et al., 2017)

- Completely eliminates recurrence and convolutions.
- Purely reliant on Self-Attention mechanisms and Feed-Forward networks.
- Structural Layout:
  - Stacked Encoder layers ($N = 6$).
  - Stacked Decoder layers ($N = 6$).
  - Residual connections and Layer Normalization.

Slide 10: Query, Key, and Value Representations

- Input matrix $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{model}}}$.
- Linear projections into Query ($\mathbf{Q}$), Key ($\mathbf{K}$), and Value ($\mathbf{V}$):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K \in \mathbb{R}^{N \times d_k}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V \in \mathbb{R}^{N \times d_v}$$

Slide 11: Scaled Dot-Product Attention Formulation

- Compute similarity between Query and Key vectors:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

- Inner dot product $\mathbf{Q}\mathbf{K}^T$ measures token affinity.
- Softmax turns raw scores into a probability distribution over sequence items.

Slide 12: The Scaling Factor $\sqrt{d_k}$

- Why divide by $\sqrt{d_k}$?
- For large dimensions d_k, inner products grow large in magnitude:

$$\text{Var}(q \cdot k) = d_k \quad \text{(assuming mean } 0 \text{ and variance } 1\text{)}$$

- Large magnitudes push the `softmax` function into regions with extremely small gradients.
- Scaling by $\sqrt{d_k}$ preserves stable gradient flow during backpropagation.

Slide 13: Multi-Head Attention (MHA) Concept

- Instead of single attention space, project queries, keys, and values h times independently.
- Allows the model to jointly attend to information from different representation subspaces at different positions.
- Example: One head focuses on syntactic dependencies, another on semantic associations.

Slide 14: Multi-Head Attention Equations

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$

$$\text{where head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

- Projection matrices: - $\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ - $\mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ - $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ - $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$

Slide 15: Position-wise Feed-Forward Networks (FFN)

- Applied to each position separately and identically:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

- Alternately expressed with GELU / ReLU activation functions.
- Inner dimensionality is expanded (typically $d_{ff} = 4 \times d_{\text{model}}$).
- Contains a significant portion of the total parameter budget.

Slide 16: Positional Encoding Motivation

- Self-attention is permutation-invariant:

$$\text{Attention}(\mathbf{PQ}, \mathbf{PK}, \mathbf{PV}) = \mathbf{P} \cdot \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

- Without positional indicators, “Dog bites man” and “Man bites dog” yield identical representations.
- Must inject explicit sequence order information into input embeddings.

Slide 17: Sinusoidal Positional Encoding

- Fixed positional encoding using sine and cosine functions:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- Allows model to easily learn to attend by relative positions, as PE_{pos+k} can be represented as a linear function of PE_{pos}.

Slide 18: Learned Positional Embeddings

- Alternative: Initialize positional embedding matrix $\mathbf{E}_{pos} \in \mathbb{R}^{N_{\max} \times d_{\text{model}}}$.
- Trained end-to-end alongside word embeddings.
- Used in models like BERT, GPT-2.
- *Trade-off:* Cannot easily extrapolate beyond maximum trained sequence length $N_{\max}$.

Slide 19: Residual Connections & Layer Normalization

- Each sub-layer (Attention, FFN) employs a residual wrapper:

$$\text{Output} = \text{LayerNorm}\big(\mathbf{x} + \text{SubLayer}\big(\mathbf{x}\big)\big)$$

- **Post-LN (Original Transformer):** $\text{LayerNorm}\big(\mathbf{x} + \text{SubLayer}\big(\mathbf{x}\big)\big)$
- **Pre-LN (Modern Practice):** $\mathbf{x} + \text{SubLayer}\big(\text{LayerNorm}\big(\mathbf{x}\big)\big)$
- Pre-LN yields substantially more stable training dynamics for deep transformers without warm-up heuristics.

Slide 20: Encoder Architecture Summary

- Input Tokens $\rightarrow$ Token Embedding + Positional Encoding.
- Stack of N identical Encoder Layers:
  - Multi-Head Self-Attention.
  - Residual Connection & LayerNorm.
  - Position-wise FFN.
  - Residual Connection & LayerNorm.
- Output: Sequence of contextual vector representations.

Slide 21: Decoder Architecture & Masked Attention

- Decoder processes target tokens.
- **Masked Multi-Head Attention:** Prevents positions from attending to subsequent (future) positions.
- Mask matrix $\mathbf{M}$:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

- Ensures autoregressive causality during generation.

Slide 22: Encoder-Decoder Cross-Attention

- Second attention sub-layer in Decoder.
- Queries ($\mathbf{Q}$) come from previous Decoder sub-layer.
- Keys ($\mathbf{K}$) and Values ($\mathbf{V}$) originate from Encoder output representations.
- Bridges source sequence information with target sequence generation.

PART III: PRETRAINING PARADIGMS AND BERT

Slide 23: The Transfer Learning Paradigm in NLP

- **Phase 1: Self-Supervised Pretraining**
  - Train on massive unlabeled text corpora (Wikipedia, BookCorpus, Common Crawl).
  - Learn language structure, grammar, world knowledge, and context semantics.
- **Phase 2: Supervised Fine-Tuning**
  - Adapt model to small labeled datasets for specific tasks (SQuAD, MNLI, SST-2).

Slide 24: Taxonomy of Pretrained Transformer Families

- **Auto-regressive / Decoder-only (e.g., GPT series):**
  - Left-to-right unidirectional context modeling. Ideal for text generation.
- **Auto-encoding / Encoder-only (e.g., BERT, RoBERTa):**
  - Bidirectional context modeling. Ideal for analysis and representation extraction.
- **Sequence-to-Sequence / Encoder-Decoder (e.g., T5, BART):**
  - Full encoder + causal decoder. Ideal for summarization and translation.

Slide 25: What is BERT?

- **BERT:** Bidirectional Encoder Representations from Transformers.
- Introduced by Devlin et al. (Google AI, 2018).
- Designed to pretrain deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers.

Slide 26: Architectural Configurations of BERT

- **BERT-Base:**
  - $L = 12$ layers (Transformer blocks), $H = 768$ hidden size, $A = 12$ attention heads.
  - Total Parameters: **110M**.
- **BERT-Large:**
  - $L = 24$ layers, $H = 1024$ hidden size, $A = 16$ attention heads.
  - Total Parameters: **340M**.

Slide 27: BERT Input Representation Scheme

- Input sequence can represent a single sentence or a pair of sentences.
- Three additive embedding components:

$$\mathbf{E}_{\text{input}} = \mathbf{E}_{\text{Token}} + \mathbf{E}_{\text{Segment}} + \mathbf{E}_{\text{Position}}$$

- **Data Flow:** $\mathbf{E}_{\text{input}}$ is fed into the first Transformer Encoder layer.

Slide 28: Special Tokens in BERT

- **[CLS] Token:**
  - Standing for “Classification”.
  - Always inserted as the first token of every sequence.
  - Final hidden state $\mathbf{h}_{[\text{CLS}]}$ serves as aggregate sequence representation.
- **[SEP] Token:**
  - Separates two distinct sentences (e.g., Question & Answer).
  - Marks the end of input sequences.

Slide 29: Segment and Position Embeddings

- **Segment Embeddings ($\mathbf{E}_{\text{Segment}}$):**
  - Distinguishes Sentence A from Sentence B in pair tasks.
  - Vector E_A added to tokens in first sentence; E_B to tokens in second sentence.
- **Learned Positional Embeddings ($\mathbf{E}_{\text{Position}}$):**
  - Index-based spatial location encoding ($pos \in [0, 512]$).

Slide 30: Subword Tokenization: WordPiece

- Solves Out-of-Vocabulary (OOV) problems and reduces vocabulary size.
- Vocabulary size typical: ~30,000 tokens.
- Frequent words kept whole (`playing`); rare words broken into subwords (`play`, `##ing`).
- Characters serve as fallback level.

Slide 31: Pretraining Task 1 - Masked Language Model (MLM)

- Standard language models only look left-to-right or right-to-left.
- Deep bidirectionality causes target tokens to “see themselves” in multi-layer attention.
- Solution: Randomly mask 15% of input tokens, then predict masked tokens using cross-entropy loss.

Slide 32: The 80-10-10 MLM Masking Recipe

- Of the 15% selected tokens:
  - **80%** of the time: Replace token with `[MASK]`.
    - *Example:* "my dog is hairy" $\rightarrow$ "my dog is `[MASK]`"
  - **10%** of the time: Replace token with random word.
    - *Example:* "my dog is hairy" $\rightarrow$ "my dog is `apple`"
  - **10%** of the time: Keep original token unchanged.
    - *Purpose:* Biases representation towards actual observed words.

Slide 33: Pretraining Task 2 - Next Sentence Prediction (NSP)

- Understands relationships between two sentences (crucial for QA and NLI).
- Binarized classification dataset constructed from pretraining corpus:
  - **50% of pairs (IsNext):** Sentence B actually follows Sentence A.
  - **50% of pairs (NotNext):** Sentence B is a random sentence from corpus.
- Final output vector of `[CLS]` used to predict binary label.

Slide 34: The Joint Pretraining Loss Function

- Total pretraining loss is the sum of MLM and NSP classification objectives:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MLM}}(\theta) + \mathcal{L}_{\text{NSP}}(\theta)$$

- Optimization via AdamW with warm-up linear learning rate decay.
- Trained on BookCorpus (800M words) and English Wikipedia (2,500M words).

PART IV: FINE-TUNING BERT FOR DOWNSTREAM TASKS

Slide 35: Fine-Tuning Philosophy

- Pretrained BERT weights provide a strong initialization point.
- Minimal task-specific architecture added on top (usually just a linear projection layer).
- **All parameters are updated jointly** during downstream fine-tuning.
- Fast convergence (typically 2-4 epochs sufficient).

Slide 36: Task Category 1 - Single Sentence Classification

- *Tasks:* Sentiment Analysis (SST-2), Linguistic Acceptability (CoLA).
- *Input:* `[CLS] Sentence [SEP]`
- *Classification Head:*

$$\mathbf{P} = \text{softmax}(\mathbf{W}_{\text{cls}}\mathbf{h}_{[\text{CLS}]} + \mathbf{b})$$

where $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{K \times H}$, K = number of classes.

Slide 37: Task Category 2 - Sentence Pair Classification

- *Tasks:* Natural Language Inference (MNLI, RTE), Semantic Textual Similarity (MRPC, QQP).
- *Input:* `[CLS] Sentence A [SEP] Sentence B [SEP]`
- *Process:* Self-attention automatically models cross-sentence interactions.
- Pass $\mathbf{h}_{[\text{CLS}]}$ into task-specific classifier layer.

Slide 38: Task Category 3 - Token-Level Sequence Labeling

- *Tasks:* Named Entity Recognition (NER), Part-of-Speech (POS) Tagging.
- *Input:* `[CLS] Tokens [SEP]`
- *Head:* Pass final hidden state of *each* token $\mathbf{h}_i$ to token classifier:

$$\mathbf{P}_i = \text{softmax}(\mathbf{W}_{\text{token}}\mathbf{h}_i + \mathbf{b})$$

- Note: Subword alignment strategy required for WordPiece outputs.

Slide 39: Task Category 4 - Question Answering (SQuAD v1.1)

- *Input:* `[CLS] Question [SEP] Passage/Context [SEP]`
- Goal: Predict start and end text spans of answer within Passage.
- Learn two vectors: Start Vector $\mathbf{S} \in \mathbb{R}^H$ and End Vector $\mathbf{E} \in \mathbb{R}^H$.
- Probability of token i being start of span:

$$P_i = \frac{\exp(\mathbf{S} \cdot \mathbf{h}_i)}{\sum_j \exp(\mathbf{S} \cdot \mathbf{h}_j)}$$

Slide 40: Fine-Tuning Hyperparameter Recommendations

- **Batch Size:** 16, 32
- **Learning Rate (AdamW):** 5e-5, 3e-5, 2e-5
- **Number of Epochs:** 2 to 4
- **Dropout Rate:** Keep default 0.1
- Linear learning rate warmup over first 10% of total training steps.

Slide 41: Optimization - AdamW Optimizer

- Standard Adam applies L2 weight decay to gradient update directly.
- **AdamW (Loshchilov & Hutter):** Decouples weight decay from gradient update computation:

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

- Essential for preventing weight decay from degrading adaptive learning rate tracking in Transformers.

Slide 42: Catastrophic Forgetting & Mitigation

- **Risk:** Fine-tuning over small datasets can overwrite pretrained representations.
- **Mitigation Techniques:**
  - Discriminative Fine-Tuning (lower learning rates for lower layers).
  - Layer-wise learning rate decay (LLRD):

$$\eta_l = \eta \cdot \gamma^{L-l}$$

  - Early stopping monitoring validation performance.

Slide 43: Practical Implementation Workflow

1. Load Pretrained Tokenizer and Base Weights (e.g., HuggingFace `transformers`).
2. Tokenize input text (add `[CLS]`, `[SEP]`, generate attention masks & token_type_ids).
3. Append linear task classification layer.
4. Set optimizer (AdamW) with weight decay exclusions (bias & LayerNorm parameters).
5. Train for 3 epochs with linear warmup scheduler.

PART V: TRANSFORMER VARIANTS & EVOLUTION

Slide 44: RoBERTa - Robustly Optimized BERT

- Liu et al. (FB AI, 2019) demonstrated BERT was significantly undertrained.
- Key Modifications:
  - Removed Next Sentence Prediction (NSP) task.
  - Dynamic Masking instead of static masking during data preprocessing.
  - Larger batch sizes (up to 8,000) and longer pretraining on 160GB text.

Slide 45: ALBERT - A Lite BERT

- Lan et al. (Google, 2019) aimed to reduce BERT parameter footprint.
- Innovations:
  - **Factorized Embedding Parameterization:** Split embedding dimension E from hidden dimension H ($E \ll H$).
  - **Cross-Layer Parameter Sharing:** Share weights across all L Transformer layers.
  - Replaced NSP with Sentence Order Prediction (SOP).

Slide 46: ELECTRA - Pretraining as Discriminator

- Clark et al. (2020) proposed **Replaced Token Detection (RTD)**.
- Generator network masks tokens and generates plausible substitutes.
- Discriminator predicts whether each token is **original or replaced**.
- Train on ALL input tokens instead of just the 15% masked tokens $\rightarrow$ Compute efficient.

Slide 47: DistilBERT - Knowledge Distillation

- Sanh et al. (HuggingFace, 2019).
- Student model trained to mimic output distribution of Teacher BERT-Base.
- Uses triple loss: Supervised Masked LM loss, Distillation loss, Cosine Embedding loss.
- 40% fewer parameters, 60% faster inference, retains 97% language understanding capabilities.

Slide 48: Long-Sequence Transformers (Longformer / BigBird)

- Standard Self-Attention scale $\mathcal{O}(N^2)$ fails for sequence length $N > 512$.
- **Longformer:** Replaces dense attention matrix with sparse attention patterns:
  - Sliding window attention (local context).
  - Dilated sliding window attention.
  - Global attention on specific pre-selected task tokens (e.g., `[CLS]`).

Slide 49: XLNet - Permutation Language Modeling

- Yang et al. (2019) combined auto-regressive and auto-encoding advantages.
- Permutation LM pretraining objective: Maximize expected log-likelihood over all possible factorization orders of input sequence.
- Uses Two-Stream Self-Attention mechanism (Query Stream & Content Stream).

Slide 50: Comparative Overview of Encoder Models

Model	Parameters	Pretraining Objective	Key Architectural Innovation
BERT	110M / 340M	MLM + NSP	Deep Bidirectional Encoder
RoBERTa	125M / 355M	Dynamic MLM	Large Batch + No NSP
ALBERT	12M / 235M	MLM + SOP	Parameter Sharing
ELECTRA	14M / 335M	RTD	Replaced Token Discriminator

PART VI: PRACTICAL HANDS-ON GUIDANCE

Slide 51: HuggingFace Pipeline Architecture

- HuggingFace `transformers` modular setup:
  - `AutoTokenizer`: Converts raw text into numerical token IDs and attention masks.
  - `AutoModelForSequenceClassification`: Pretrained backbone + linear classification head.
  - `Trainer` API / PyTorch Training Loop: Controls gradient computation and optimization steps.

Slide 52: Data Preprocessing & Attention Masks

- **Token IDs:** Array of vocabulary indices.
- **Attention Mask:** Binary vector flagging real tokens (1) vs. padding tokens (0).
  - Prevents attention mechanism from calculating scores for padding elements.
- **Token Type IDs:** Distinguishes Sentence A (0) from Sentence B (1).

Slide 53: PyTorch Dataset Implementation

```
class TextDataset(torch.utils.data.Dataset):
    def __init__(self, texts, labels, tokenizer,
    max_len=128):
        self.encodings = tokenizer(
            texts, truncation=True, padding=True,
            max_length=max_len, return_tensors="pt"
        )
        self.labels = torch.tensor(labels)

    def __getitem__(self, idx):
        item = {k: v[idx] for k, v in
self.encodings.items()}
        item['labels'] = self.labels[idx]
        return item
```

Slide 54: Building Custom Classifier Heads

```
class CustomBERTClassifier(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.bert =
        AutoModel.from_pretrained('bert-base-uncased')
        self.drop = nn.Dropout(p=0.3)
        self.out =
        nn.Linear(self.bert.config.hidden_size,
        num_classes)

    def forward(self, input_ids, attention_mask):
        outputs = self.bert(input_ids=input_ids,
attention_mask=attention_mask)
        pooled_output = outputs.pooler_output
        return self.out(self.drop(pooled_output))
```

Slide 55: Parameter Efficient Fine-Tuning (PEFT)

- Fine-tuning all 110M+ parameters for every downstream task becomes expensive.
- **PEFT Methods:** Freeze backbone weights; train only a tiny set of additive parameters.
- Prominent paradigms:
  - Adapters (Houlsby et al.).
  - Prefix Tuning.
  - LoRA (Low-Rank Adaptation).

Slide 56: Low-Rank Adaptation (LoRA)

- Freezes pretrained weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$.
- Decomposes weight update $\Delta\mathbf{W}$ into two low-rank matrices $\mathbf{A}$ and $\mathbf{B}$:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}$$

  where $\mathbf{B} \in \mathbb{R}^{d \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times k}$, and rank $r \ll \min(d, k)$.
- Reduces trainable parameter count by $> 99\%$ while maintaining performance.

Slide 57: Hardware Acceleration & Mixed Precision

- **FP16 / BF16 Automatic Mixed Precision (AMP):**
  - Keeps master weights in FP32; computes forward/backward passes in FP16/BF16.
  - Halves GPU memory requirements; accelerates tensor core matrix multiplications.
- Gradient Accumulation: Simulates large batch sizes over hardware with memory constraints.

Slide 58: Quantization Techniques (INT8 / INT4)

- Post-Training Quantization (PTQ) & Quantization-Aware Training (QAT).
- Reduces float weight representations into low-bit integers:

$$\mathbf{Q} = \text{round}\left(\frac{\mathbf{W}}{S}\right) + Z$$

- Enables deployment of fine-tuned models on edge devices and resource-limited servers.

PART VII: EVALUATION AND DIAGNOSTICS

Slide 59: Standard NLP Benchmark Suites

- **GLUE (General Language Understanding Evaluation):**
  - Collection of 9 sentence-level understanding tasks (SST-2, MNLI, QQP, RTE, etc.).
- **SuperGLUE:** Harder benchmark set designed to test complex reasoning, coreference resolution, and causal inference.

Slide 60: Error Analysis & Diagnostic Probing

- How do we know what fine-tuned BERT actually learns?
- **Probing Tasks:** Train lightweight linear classifiers on frozen intermediate representations to inspect syntactic/semantic awareness.
- Findings: Lower layers capture surface/syntactic patterns; higher layers encode semantic relationships.

Slide 61: Attention Map Visualization

- Inspect head-level attention weights α_{ij}.
- Tools: BertViz.
- Common Attention Patterns:
  - **Positional:** Attending to immediate adjacent tokens ($i+1$ or $i-1$).
  - **Delimiting:** High focus on [CLS] and [SEP] tokens.
  - **Lexical/Syntactic:** Attending from verbs to direct objects.

Slide 62: Handling Class Imbalance in Fine-Tuning

- Common issue in real-world classification tasks.
- Strategies:
  - Weighted Cross-Entropy Loss:

$$\mathcal{L} = -\sum w_c \cdot y_c \log(\hat{y}_c)$$

  - Focal Loss: Down-weights well-classified examples to focus on hard samples.
  - Oversampling / Undersampling token sequences.

Slide 63: Overfitting Signs & Regularization

- **Symptoms:** Validation loss increases while training loss continues decreasing.
- **Remedies:**
  - Increase Dropout rate on classification head.
  - Apply Weight Decay ($\lambda = 0.01$).
  - Early Stopping based on Validation F1-score.
  - Adversarial Training (e.g., FGM, PGD perturbation on embeddings).

PART VIII: BEYOND TEXT - MULTIMODAL & ADVANCED APPLICATIONS

Slide 64: Domain-Specific BERT Variants

- Standard BERT trained on general domain text (Wikipedia).
- **BioBERT:** Pretrained on PubMed abstracts & PMC full-text articles.
- **ClinicalBERT:** Pretrained on ICU clinical notes (MIMIC-III).
- **FinBERT:** Pretrained on financial news and corporate filings.
- Demonstrates superiority over general BERT on specialized domain jargon.

Slide 65: Multilingual Representations - mBERT & XLM-R

- **mBERT:** Single BERT model pretrained on Wikipedia of 104 languages simultaneously.
- **XLM-RoBERTa:** Trained on 2.5TB of filtered CommonCrawl data covering 100 languages.
- Enables zero-shot cross-lingual transfer: Fine-tune in English $\rightarrow$ Test directly in Vietnamese.

Slide 66: Vision Transformers (ViT)

- Dosovitskiy et al. (2020) adapted Transformer directly to images.
- Image $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ split into non-overlapping patches $p \times p$.
- Patches flattened and linearly projected into vector embeddings.
- Processed by standard Encoder-only Transformer backbone.

Slide 67: Multimodal Transformers (CLIP & Vision-Language)

- Joint text-image modeling using dual Transformers.
- **CLIP (OpenAI):** Contrastive pretraining between Image Encoder and Text Encoder.
- Maximizes cosine similarity of positive (Image, Text) pairs while minimizing negative pairs.

PART IX: COMPREHENSIVE RECAP & FUTURE DIRECTIONS

Slide 68: Summary of Pretraining vs. Fine-Tuning

- Pretraining acquires broad linguistic and contextual priors via self-supervision.
- Fine-Tuning redirects network representations toward specific task classification boundaries.
- Compute cost ratio: Pretraining $\approx 99\%$, Fine-Tuning $\approx 1\%$.

Slide 69: Key Takeaways on Architecture Selection

- Choose **Encoder-only (BERT/RoBERTa)** for classification, tagging, and extraction.
- Choose **Decoder-only (GPT-3/4, LLaMA)** for text generation and prompt-based tasks.
- Choose **Encoder-Decoder (T5/BART)** for sequence transformation (Summarization, Translation).

Slide 70: Open Challenges in Transformer Research

- Quadratic memory scale of attention mechanisms ($\mathcal{O}(N^2)$).
- Interpretability and Mechanistic Interpretability.
- Robustness against adversarial input perturbations.
- High computational requirements for edge deployment.

PART X: ADVANCED TOPICS AND RESEARCH FRONTIERS

Slide 71: Masked Autoencoders (MAE) in Computer Vision

- He et al. (2021) applied BERT-style masking to Vision Transformers.
- High masking ratio (75%) applied to image patches.
- Lightweight decoder reconstructs missing pixel values.
- Proves self-supervised autoencoding scales across modalities.

Slide 72: Sentence-BERT (SBERT) & Bi-Encoders

- Standard BERT requires both sentences passed together $\rightarrow$ Computationally heavy for search ($\mathcal{O}(N^2)$).
- **SBERT (Reimers & Gurevych):** Uses Siamese network structure to derive semantically meaningful fixed-length sentence embeddings.
- Enables fast semantic search via Cosine Similarity and Vector Databases (FAISS, Milvus).

Slide 73: Cross-Encoders vs. Bi-Encoders

- **Bi-Encoder:** Encodes query and document separately into static vectors. Fast lookup, lower accuracy.
- **Cross-Encoder:** Encodes query and document jointly through full attention layers. Slow lookup, superior accuracy.
- Modern Retrieval Reranking Pipeline: Bi-Encoder for retrieval $\rightarrow$ Cross-Encoder for reranking.

Slide 74: Gradient Checkpointing for Long Inputs

- Stores only a subset of intermediate activation tensors during forward pass.
- Recomputes missing activations dynamically during backward pass.
- Trades computational time (+20% compute overhead) for significant VRAM savings (up to 60-70%).

Slide 75: Knowledge Distillation Formulations

- Minimizes Kullback-Leibler (KL) Divergence between Teacher logits $\mathbf{z}^T$ and Student logits $\mathbf{z}^S$:

$$\mathcal{L}_{\text{KD}} = T^2 \cdot D_{\text{KL}} \left(\text{softmax} \left(\frac{\mathbf{z}^T}{T} \right) \parallel \text{softmax} \left(\frac{\mathbf{z}^S}{T} \right) \right)$$

where T represents the Softmax Temperature hyperparameter.

Slide 76: Modern Prompt Tuning vs. Fine-Tuning

- **Full Fine-Tuning:** Updates all model parameters per task.
- **Prefix / Prompt Tuning:** Prepends trainable continuous virtual tokens to Key and Value matrices in each Transformer layer.
- Keeps base model entirely frozen while achieving competitive downstream performance.

Slide 77: Ethics, Bias, and Toxicity in Pretrained Models

- Pretraining datasets contain implicit social biases, stereotypes, and toxic language.
- Fine-tuning on task datasets does not automatically erase systemic pretraining biases.
- Mitigation strategies: Bias probing, counterfactual data augmentation, and alignment tuning (RLHF/DPO).

Slide 78: Summary Checklist for Practitioners

1. Select appropriate pretrained model variant (RoBERTa for general NLP, BioBERT for biomedical).
2. Ensure subword token alignment matches label alignment for sequence tagging.
3. Apply AdamW with weight decay exclusions for Bias and LayerNorm.
4. Utilize Mixed Precision (AMP) and low learning rates (2×10^{-5}).
5. Evaluate thoroughly on validation set with appropriate task-specific metrics.

Slide 79: References & Key Literature

1. Vaswani et al. (2017). *Attention Is All You Need*. NeurIPS.
2. Devlin et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.
3. Liu et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv.
4. Hu et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR.

Slide 80: Q&A and Concluding Remarks

Thank You for Your Attention!

Questions & Discussion